

UNIVERSIDADE FEDERAL DO AGRESTE DE PERNAMBUCO

Curso de Ciência da Computação

Disciplina: Programação Orientada a Objetos

LibraryHub

Relatório de Desenvolvimento - Terceira VA

Reservas e multas: fila de espera e cobrança por atraso ou dano

Lucas Tchaikovsky dos Santos Rodrigues

Professor: Igor Medeiros Vanderlei

Garanhuns, PE - Setembro de 2026

Relatório de Desenvolvimento — Terceira VA

Disciplina: Programação Orientada a Objetos

Curso: Ciência da Computação — UFAPE

Professor: Igor Medeiros Vanderlei

Estudante: Lucas Tchaikovsky dos Santos Rodrigues — GitHub Lucas210301

Projeto: LibraryHub — gestão de empréstimos de biblioteca

Repositório: <https://github.com/Lucas210301/libraryhub>

Branch da terceira VA: terceira-va

Período coberto: 04/09/2026 a 13/09/2026

Relatório restrito ao desenvolvimento da última semana. A base da segunda VA — membros, itens e empréstimos — é citada apenas quando necessário para explicar as decisões novas.

1. Apresentação do Problema

O LibraryHub controla o acervo de uma biblioteca e os empréstimos feitos aos seus membros. A entrega anterior já registrava membros, itens (livros e revistas) e empréstimos, com prazo calculado pelo tipo do item e limite de três empréstimos ativos por membro. Duas situações reais ainda não tinham tratamento, e foram o objeto desta VA.

A primeira é a **disputa por um item já emprestado**: quem procurasse um livro em uso não tinha como registrar interesse, o que favorecia quem aparecia primeiro por acaso, não quem esperava há mais tempo. A segunda é a **falta de consequência para a má devolução**: o sistema aceitava um item semanas atrasado, ou danificado, como aceitaria uma devolução em dia, e o membro podia pegar outro em seguida.

Funcionalidades implementadas:

1. **Reserva de item** — o membro entra na fila de um item emprestado; a reserva pode ser cancelada e é atendida quando aquele membro pega o item.
2. **Fila respeitada no empréstimo** — o primeiro da fila tem prioridade e um terceiro membro é impedido de levar o item enquanto a reserva estiver ativa.
3. **Multa por atraso** — gerada automaticamente na devolução fora do prazo, proporcional aos dias.
4. **Multa por dano** — registrada pela equipe, com valor definido pela gravidade.
5. **Bloqueio por débito** — membro com multa em aberto não faz novo empréstimo nem é removido.
6. **Pagamento e saldo devedor** — a tela de multas mostra o total em aberto e permite dar baixa.

2. Modelo de Classes

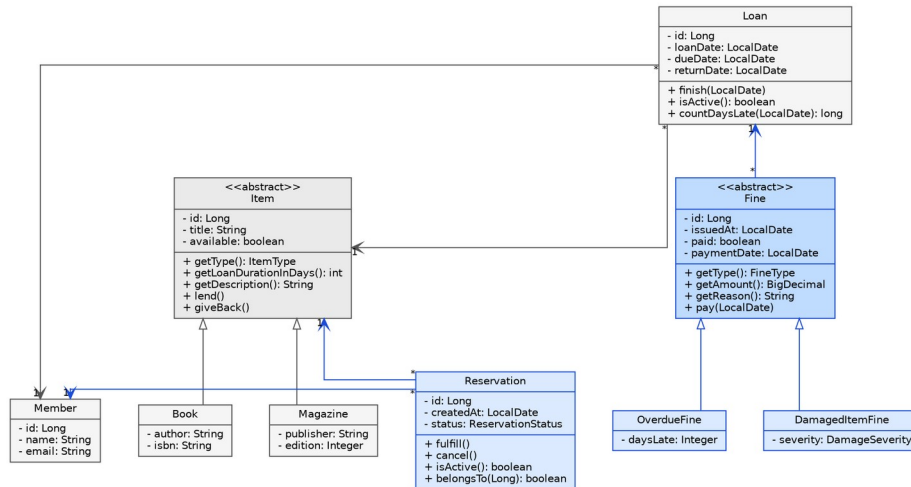


Figura 1 — Diagrama de classes. Em azul, as classes criadas nesta VA; em cinza, as da entrega anterior, mostradas apenas pelos relacionamentos.

Decisões principais. `Reservation` é uma classe própria, e não um campo `reservedBy` em `Item`: como campo, quebraria assim que a biblioteca quisesse mais de uma pessoa na fila e perderia a data de entrada, que é o critério de ordenação. Seu estado fica em um enum (`ACTIVE`, `FULFILLED`, `CANCELLED`) em vez de dois booleanos, que permitiriam o estado impossível de uma reserva atendida e cancelada ao mesmo tempo.

`Fine` é abstrata com duas subclasses porque as duas multas compartilham identidade, data, situação de pagamento e a ligação com o empréstimo, mas calculam o valor de formas incompatíveis: uma multiplica dias de atraso por uma diária, a outra consulta uma tabela de gravidade. A hierarquia evita uma classe única com `if` sobre um campo `tipo` e metade dos atributos nulos. O valor é derivado por `getAmount()` e não armazenado, para que não possa divergir da regra que o gerou; `Fine` aponta para `Loan`, que já carrega membro e item, evitando dois caminhos para a mesma informação.

As invariantes ficam nas próprias classes básicas: `Reservation.fulfill()`, `cancel()` e `Fine.pay()` verificam a situação atual antes de mudá-la e nenhuma expõe setter para o estado — o mesmo padrão de `Item.lend()`.

Conceito	Onde aparece
Abstração	<code>Fine</code> define o que toda multa é e sabe fazer, sem dizer como o valor é calculado
Herança	<code>OverdueFine</code> e <code>DamagedItemFine</code> estendem <code>Fine</code>
Polimorfismo	<code>getAmount()</code> , <code>getType()</code> e <code>getReason()</code> têm implementação própria em cada subclasse; <code>FineService.totalUnpaid()</code> e <code>FineConverter</code> operam sobre <code>Fine</code> sem nenhum <code>instanceof</code>
Encapsulamento	<code>status</code> , <code>paid</code> e <code>paymentDate</code> são privados e só mudam por métodos que validam antes

Conceito	Onde aparece
Associação	Reservation associa Member e Item; Fine associa Loan
Coesão	Reservation.requireActive() e Facade.fulfillReservationQueue() isolam uma decisão cada; cada cadastro trata de um assunto só

3. Arquitetura da Solução

O módulo foi encaixado nas camadas existentes, sem estrutura paralela, e nenhuma tecnologia nova foi introduzida: Java 21 com Spring Boot 3.5, Spring Data JPA sobre PostgreSQL, Bean Validation, front end em Next.js com axios e daisyUI, e testes com JUnit 5, Mockito, MockMvc, H2 e Playwright.



Figura 2 — Camadas do sistema. Em azul, os componentes criados nesta VA. Cada camada só conhece a de baixo por interface.

Camada	Artefatos criados
Classes básicas (business.model)	Reservation, ReservationStatus, Fine, OverdueFine, DamagedItemFine, FineType, DamageSeverity
Coleção de dados (data)	IReservationRepository, IFineRepository
Coleção de negócio (business.service)	IReservationService, ReservationService, IFineService, FineService
Fachada (business.facade)	Sete operações novas em IFacade e Facade
Comunicação (communication)	ReservationController, FineController, quatro DTOs, dois conversores
Exceções (exception)	Nove classes novas, herdando de BusinessException
Interface gráfica (frontend)	Telas de reservas e multas, dois server actions, dois formulários

Um pedido de reserva percorre: navegador → server action → POST /reservations em JSON → ReservationController → IFacade.reserveItem → cadastros de membros, itens e reservas → JPA → PostgreSQL, com o ReservationConverter transformando a entidade em DTO na volta. Reservation tem tabela própria; Fine usa herança de tabela única, com a coluna fine_type separando OVERDUE de DAMAGE — mesma estratégia já usada em Item. Foram criadas nove rotas: quatro em /reservations (listar, buscar, criar, cancelar) e cinco em /fines (listar, buscar, saldo em aberto, registrar dano, pagar), usando os status 200, 201, 400, 404 e 409.

Regra de negócio	Camada	Exceção
Reserva atendida ou cancelada	Classe básica	ReservationNotActiveException

Regra de negócio	Camada	Exceção
não muda mais de estado		
Multa paga não é paga de novo	Classe básica	FineAlreadyPaidException
Membro não tem duas reservas ativas do mesmo item	Coleção de negócio	DuplicatedReservationException
Empréstimo tem no máximo uma multa por dano	Coleção de negócio	DuplicatedFineException
Item disponível é emprestado, não reservado	Fachada	ItemAvailableException
Item reservado por outro membro não é emprestado	Fachada	ItemReservedByAnotherMemberException
Membro com multa em aberto não pega item nem é removido	Fachada	MemberWithUnpaidFinesException

As três últimas ficaram na fachada porque nenhuma pode ser respondida por um cadastro sozinho: cruzam, respectivamente, itens e reservas; membros, itens e reservas; e membros, empréstimos e multas.

4. Dificuldades Encontradas

Onde colocar a regra da fila. A primeira versão verificava a reserva dentro de `LoanService`, o que obrigava o cadastro de empréstimos a conhecer o repositório de reservas. A regra foi movida para o método privado `Facade. fulfillReservationQueue`, e cada cadastro voltou a tratar de um assunto só.

Ordem de remoção e chaves estrangeiras. Remover um membro passou a violar integridade referencial. A limpeza foi ordenada na fachada — reservas, multas, empréstimos e por último o membro — dentro de um método com `@Transactional(rollbackFor = Exception.class)`, necessário porque as exceções do projeto são checadas e o Spring não desfaz a transação para elas por padrão.

Testar multa por atraso. Como `dueDate` vem de `LocalDate.now()`, não é possível criar pela fachada um empréstimo já vencido. Em vez de abrir um setter só para o teste, `Loan.countDaysLate` e `OverdueFine.getAmount` são testados no nível unitário com datas fixas, e o teste de integração usa a multa por dano, que não depende de data, para exercitar o bloqueio do devedor.

Ambiente. A maior parte do tempo perdido não foi em código de negócio. A porta 5432 estava ocupada por um PostgreSQL nativo, então o container Docker nunca publicava a porta, com o sintoma enganoso de o container aparecer como ativo; a solução foi usar a instalação nativa e criar ali o banco `libraryhub`, que também não existia. Um processo `next dev` travado segurava a porta 3000 havia mais de seis horas, e os navegadores do Playwright nunca tinham sido baixados — os dois faziam a suíte travar sem mensagem.

Confiabilidade dos testes de interface. `getByRole("alert")` capturava, além do banner de erro do formulário, um elemento que o próprio Next.js insere em toda página; a correção foi

extrair o banner para o componente `ErrorAlert`, reutilizado por todos os formulários, e marcá-lo com um `data-testid`. As fixtures geravam registros ilegíveis como `Playwright Book 1789093254506` e foram reescritas a partir de dados reais de acervo. O seletor de membro mostrava só o nome, tornando homônimos indistinguíveis, e passou a exibir o e-mail. Com dois workers a suíte alternava entre passar e falhar, porque os testes compartilham o banco, e passou a rodar em série.

Verificação	Resultado
<code>mvn test</code>	55 testes, 55 aprovados, BUILD SUCCESS
<code>npm run lint</code>	sem erros
<code>npm run build</code>	compila, sem o aviso de lockfile
<code>npx playwright test</code>	4 aprovados, 0 falhos, 44,6 s

A estabilização consumiu 9 commits, 13 arquivos e um saldo de +229/−87 linhas, sem nenhuma alteração no back end Java — indício de que o problema estava na configuração e no acoplamento dos testes de interface, não nas regras de negócio.

5. Matriz de Responsabilidades

Projeto individual, conforme o item 1 das orientações. Todas as contribuições estão no repositório `Lucas210301/libraryhub`, branch `terceira-va`.

Membro	Funcionalidade	Requisito atendido	Artefato desenvolvido	Evidência no Git
Lucas	Reserva	Classe básica de negócio (1/2)	<code>business/model/Reservation.java</code> , <code>ReservationStatus.java</code>	1e44b23
Lucas	Multa	Classe básica de negócio (2/2)	<code>business/model/Fine.java</code> , <code>OverdueFine.java</code> , <code>DamagedItemFine.java</code> , <code>FineType.java</code> , <code>DamageSeverity.java</code>	1e44b23
Lucas	Reserva	Coleção de dados (1/2)	<code>data/IReservationRepository.java</code>	1e44b23
Lucas	Multa	Coleção de dados (2/2)	<code>data/IFineRepository.java</code>	1e44b23
Lucas	Reserva	Coleção de negócio (1/2)	<code>business/service/IReservationService.java</code> , <code>ReservationService.java</code>	1e44b23
Lucas	Multa	Coleção de negócio (2/2)	<code>business/service/IFineService.java</code> , <code>FineService.java</code>	1e44b23
Lucas	Reserva e multa	Exceção em classe básica	<code>ReservationNotActiveException</code> em <code>Reservation. fulfill/cancel;</code>	1e44b23

Membro	Funcionalidade	Requisito atendido	Artefato desenvolvido	Evidência no Git
Lucas	Reserva e multa	Exceção em coleção de negócio	FineAlreadyPaidException em Fine.pay DuplicatedReservationException em ReservationService.create; DuplicatedFineException em FineService.registerDamage	1e44b23
Lucas	Fila e débito	Exceção em fachada	ItemAvailableException, ItemReservedByAnotherMemberException, MemberWithUnpaidFinesException	1e44b23
Lucas	Reserva e multa	Teste unitário (15 casos)	business/model/ ReservationTest.java, FineTest.java, business/service/ReservationServiceTest.java	1e44b23
Lucas	Fila e débito	Teste de integração (8 casos)	integration/ ReservationFineIntegrationTest.java	1e44b23
Lucas	Reserva	Teste de API (6 casos)	api/ ReservationControllerApiTest.java	1e44b23
Lucas	Reserva	Teste de interface	frontend/tests/ reservations.spec.js	1e44b23, daa0cb3, 9aab374, e1edd98
Lucas	Reserva e multa	Interface gráfica dos casos de uso	frontend/src/app/ reservations, frontend/src/app/fines, ReservationForm.js, DamageFineForm.js	1e44b23, 7ee3e00, 93e6adf

Requisitos gerais. Herança e polimorfismo em Fine e suas subclasses, com `getAmount()`, `getType()` e `getReason()` resolvidos em tempo de execução e `FineService.totalUnpaid()` somando sem conhecer a subclasse. Encapsulamento em `status`, `paid` e `paymentDate`, sem setter e alterados só por métodos que validam. Associações de `Reservation` para `Member` e `Item`, e de `Fine` para `Loan`. Interfaces entre camadas em `IReservationService`, `IFineService`, `IReservationRepository`, `IFineRepository` e nas novas assinaturas de `IFacade`. Organização em pacotes conforme a seção 3, e documentação (`README.md`, `docs/ARQUITETURA.md`) revisada em 2cb8ea5 e 6c2b69d.

Commits. Onze commits com o usuário Lucas210301: b9485db e 1e44b23 em 10/09, que trouxeram a estrutura do repositório e o back end, os testes e as telas, incluindo o módulo desta VA; e nove em 11/09 — 6c2b69d, b72d3ee, 2c62c0c, 7ee3e00, daa0cb3, 9aab374,

93e6adf, e1edd98 e 2cb8ea5 — correspondentes à estabilização descrita na seção 4. A matriz acima indica o caminho de arquivo de cada requisito, permitindo a verificação por arquivo além do hash.

6. Conclusões

As seis funcionalidades da seção 1 foram implementadas e estão cobertas por testes nos quatro níveis exigidos. Os mínimos individuais foram atendidos a partir de duas classes que não existiam no projeto: quatro classes básicas novas, duas coleções de dados, duas coleções de negócio, nove exceções nos três níveis, cinco classes de teste no back end e um arquivo de teste de interface. A validação final fechou com 55 de 55 testes de back end aprovados, lint e build do front end sem erros e a suíte de interface passando em 44,6 segundos — situação diferente da do início da semana, quando ela travava sem produzir resultado. Para a demonstração, o banco foi carregado com um acervo realista: 8 membros, 12 livros, 4 revistas, 4 empréstimos e 3 reservas.

O objetivo era fechar as duas lacunas sem reestruturar o que já funcionava, e isso foi cumprido: as alterações em código pré-existente se resumiram ao acréscimo de `countDaysLate` em `Loan`, às novas verificações em `Facade.registerLoan` e `Facade.removeMember`, e aos `catch` correspondentes em dois controllers. Nenhuma classe da entrega anterior precisou ser reescrita.

Aprendizados. O critério para decidir entre cadastro e fachada ficou claro na prática: se a resposta a uma pergunta exige mais de uma coleção, a regra não pertence a nenhum cadastro isoladamente — a primeira versão da fila violava isso, e o sintoma apareceu como acoplamento antes de qualquer erro de execução. A hierarquia de `Fine` mostrou que polimorfismo útil não se cria para “usar herança”: ela surgiu porque duas regras de cálculo disputavam o mesmo método, e o resultado é que um terceiro tipo de multa não exige tocar em nenhum `if` existente. E, nos testes de interface, um teste que depende de seletor genérico ou de dados aleatórios não falha por causa do sistema, falha por causa do próprio teste.

Limitações e continuidade. O total em aberto é somado em memória, porque o valor da multa é derivado; a multa por atraso só é gerada na devolução, então um item nunca devolvido não gera cobrança; a fila de reserva não tem prazo de validade; a suíte de interface não remove o que cria; e o repositório ainda não tem integração contínua. Como continuidade, prazo de retirada para a reserva, rotina diária gerando multa para empréstimos vencidos, limpeza das fixtures ao fim da suíte, um workflow de CI com as quatro verificações e um relatório em PDF do saldo devedor por membro, que atenderia um dos pontos extras das diretrizes.